

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

*I _Pusalapati Chandu(192372119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *Pchandu*

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Password Hashing with Salting and Key Stretching

Design and implement a program in Python to store and verify passwords using salted hashing (PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Python Code

```
import hashlib
import os
import binascii
import time
```

```

def hash_password(password, iterations=100000):
    salt = os.urandom(16)
    key = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'),
    salt, iterations)
    return binascii.hexlify(salt).decode(),
    binascii.hexlify(key).decode()

def verify_password(password, salt_hex, key_hex, iterations=100000):
    salt = binascii.unhexlify(salt_hex)
    key = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'),
    salt, iterations)
    return binascii.hexlify(key).decode() == key_hex

if __name__ == "__main__":
    password = "MySecurePass123"
    start = time.time()
    salt, key = hash_password(password)
    end = time.time()
    print("Salt:", salt)
    print("Hashed Key:", key)
    print("Time taken to hash (s):", end - start)
    print("Verify correct password:", verify_password(password, salt,
    key))
    print("Verify wrong password:", verify_password("WrongPass", salt,
    key))

```

Step-by-Step Explanation

Step 1: Import hashlib for PBKDF2, os for generating random salt, binascii for hex conversion, and time for measuring performance.

Step 2: Write hash_password() which generates a random 16-byte salt using os.urandom() so every password gets a unique hash even if two users choose the same password.

Step 3: Apply hashlib.pbkdf2_hmac() with SHA-256, the password, the salt, and 100000 iterations; the high iteration count is the key-stretching step that slows down each guess.

Step 4: Return the salt and derived key as hex strings so they can be stored safely in a database.

Step 5: Write verify_password() which recomputes the hash using the stored salt and the same iteration count, then compares it to the stored key.

Step 6: In the main block, hash a sample password, print the salt, hashed key, and time taken, then verify with both the correct and an incorrect password to show the mismatch is detected.

Step 7: Analyze results: salting defeats precomputed rainbow tables because each password now needs its own table, and key stretching (many iterations) makes brute-force attacks thousands of times slower, at the cost of a small, controllable delay per login.

OUTPUT:

```

PS C:\Users\CHANDU\OneDrive\Documents\crypto codes\ass codes> python exp1.py
Salt: 65eed24e4105088a0943c86da13ee895
Hashed Key: a8e06ea7f668c0b36bbded622ab640d395fbb57a496353af16358089783c52a8
Time taken to hash (s): 0.0753028392791748
Verify correct password: True
Verify wrong password: False

```

Experiment 2: Implementation of HMAC for Message Integrity

Develop a Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

Python Code

```

import hmac
import hashlib

def generate_hmac(key, message):
    return hmac.new(key.encode(), message.encode(),
hashlib.sha256).hexdigest()

def verify_hmac(key, message, received_hmac):
    expected_hmac = generate_hmac(key, message)
    return hmac.compare_digest(expected_hmac, received_hmac)

if __name__ == "__main__":
    secret_key = "shared_secret_key"
    message = "Transfer $500 to Account 12345"
    tag = generate_hmac(secret_key, message)
    print("Original Message:", message)
    print("Generated HMAC:", tag)
    print("Verify unmodified message:", verify_hmac(secret_key,
message, tag))
    tampered_message = "Transfer $5000 to Account 12345"
    print("Verify tampered message:", verify_hmac(secret_key,
tampered_message, tag))

```

Step-by-Step Explanation

Step 1: Import the hmac module, which implements the HMAC construction, and hashlib to supply the SHA-256 digest function.

Step 2: Write generate_hmac() that calls hmac.new() with the shared secret key, the message, and hashlib.sha256, then returns the resulting tag as a hex digest.

Step 3: Write `verify_hmac()` that recomputes the HMAC on the receiver's side and compares it to the received tag using `hmac.compare_digest()`, which performs a constant-time comparison to avoid timing attacks.

Step 4: In the main block, generate an HMAC for a sample banking message and print it alongside the original message.

Step 5: Verify the HMAC against the untouched message to confirm it passes, then verify it again against a tampered version of the message to show the mismatch is caught.

Step 6: Analyze results: unlike a plain hash, which anyone can recompute from the message alone, HMAC mixes in a secret key so only parties holding that key can generate or validate a correct tag; this proves both integrity (the message was not altered) and authenticity (it came from someone who knows the key).

OUTPUT:

```
PS C:\Users\CHANDU\OneDrive\Documents\crypto codes\ass codes> python exp2.py
Original Message: Transfer $500 to Account 12345
Generated HMAC: b3932713f08c8749f50792ceaa4c90be7369dc3106258d6deb2ababd154bde85
Verify unmodified message: True
Verify tampered message: False
```

Experiment 3: Simulation of Certificate Chain Validation

Create a Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Python Code

```
import hashlib

class Certificate:
    def __init__(self, subject, issuer, public_key, signature):
        self.subject = subject
        self.issuer = issuer
        self.public_key = public_key
        self.signature = signature

    def sign(data, private_key):
        return hashlib.sha256((data + private_key).encode()).hexdigest()

    def verify_signature(cert, issuer_public_key):
        expected = sign(cert.subject + cert.issuer, issuer_public_key)
        return expected == cert.signature

root_priv, root_pub = "ROOT_PRIV", "ROOT_PUB"
```

```

inter_priv, inter_pub = "INTER_PRIV", "INTER_PUB"
end_priv, end_pub = "END_PRIV", "END_PUB"

root_cert = Certificate("RootCA", "RootCA", root_pub, sign("RootCA" +
"RootCA", root_priv))
inter_cert = Certificate("IntermediateCA", "RootCA", inter_pub,
sign("IntermediateCA" + "RootCA", root_priv))
end_cert = Certificate("example.com", "IntermediateCA", end_pub,
sign("example.com" + "IntermediateCA", inter_priv))

revoked_certs = ["example.com"]

def validate_chain(cert, chain, revoked):
    if cert.subject in revoked:
        return False, f"{cert.subject} is revoked"
    if cert.subject == cert.issuer:
        return True, "Root CA reached, chain trusted"
    issuer_cert = next((c for c in chain if c.subject == cert.issuer),
None)
    if issuer_cert is None:
        return False, "Issuer certificate not found"
    if not verify_signature(cert, issuer_cert.public_key):
        return False, "Signature verification failed"
    return validate_chain(issuer_cert, chain, revoked)

chain = [root_cert, inter_cert, end_cert]
result, message = validate_chain(end_cert, chain, revoked_certs)
print("Validation Result:", result)
print("Message:", message)

```

Step-by-Step Explanation

Step 1: Define a Certificate class holding subject, issuer, public key, and signature, mirroring the fields used in a real X.509 certificate.

Step 2: Write sign() to simulate signing by hashing the certificate's data together with the issuer's private key, and verify_signature() to recompute that hash with the issuer's public key and compare it.

Step 3: Build a three-level chain: a self-signed Root CA certificate, an Intermediate CA certificate signed by the root, and an end-entity certificate (example.com) signed by the intermediate.

Step 4: Maintain a revoked_certs list acting as a simplified Certificate Revocation List (CRL).

Step 5: Write validate_chain() which walks up the chain recursively: it first checks revocation, then checks if the certificate is self-signed (the root, meaning trust is established), otherwise it finds the issuer's certificate in the chain and verifies the signature before recursing upward.

Step 6: Run validate_chain() on the end-entity certificate and print whether the chain is trusted along with the reason.

Step 7: Analyze results: trust flows from the root CA downward because each certificate's signature can only be verified with its issuer's public key, so the whole chain is trustworthy only if every link verifies and the certificate is not revoked; a broken signature or a revoked entry anywhere in the chain causes validation to fail.

OUTPUT:

```
PS C:\Users\CHANDU\OneDrive\Documents\crypto codes\ass codes> python exp3.py
Validation Result: False
Message: example.com is revoked
```

Experiment 4: Replay Attack Simulation and Prevention

Develop a Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

Python Code

```
import time
import uuid

used_nonces = set()

def create_request(user, action):
    nonce = str(uuid.uuid4())
    timestamp = time.time()
    return {"user": user, "action": action, "nonce": nonce,
            "timestamp": timestamp}

def process_request(request, window=30):
    current_time = time.time()
    if request["nonce"] in used_nonces:
        return False, "Replay detected: nonce already used"
    if current_time - request["timestamp"] > window:
        return False, "Replay detected: request expired"
    used_nonces.add(request["nonce"])
    return True, "Request accepted"

if __name__ == "__main__":
    req = create_request("alice", "withdraw_1000")
    print(process_request(req))
    print(process_request(req))
    old_req = create_request("alice", "withdraw_1000")
    old_req["timestamp"] -= 60
```



```
print(process_request(old_req))
```

Step-by-Step Explanation

Step 1: Import time for timestamps and uuid to generate a unique nonce for every request, then create used_nonces as a set that tracks which nonces have already been consumed.

Step 2: Write create_request() which builds a request dictionary containing the user, the action, a fresh random nonce, and the current timestamp, simulating a legitimate client request.

Step 3: Write process_request() which first rejects the request if its nonce is already present in used_nonces (this stops an attacker from resending a captured request), and also rejects it if the timestamp is older than a defined window (this stops an attacker from replaying an old but never-before-seen request).

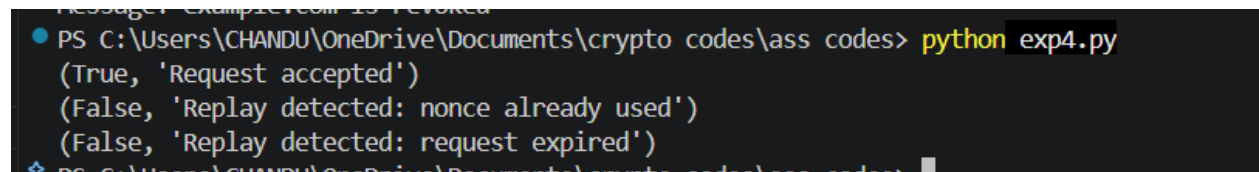
Step 4: If both checks pass, add the nonce to used_nonces and accept the request.

Step 5: In the main block, send a legitimate request and print the result, then immediately resend the identical request to simulate an attacker replaying captured traffic and show it is rejected.

Step 6: Create a third request with its timestamp shifted 60 seconds into the past to simulate an expired/delayed replay and show it is also rejected.

Step 7: Analyze results: combining a one-time nonce with a timestamp window closes both replay paths (exact resend and delayed resend), which significantly raises security; the limitation is that the server must store nonces for the length of the window and all parties need loosely synchronized clocks, adding memory and clock-skew management overhead.

OUTPUT:



```
PS C:\Users\CHANDU\OneDrive\Documents\crypto codes\ass codes> python exp4.py
(True, 'Request accepted')
(False, 'Replay detected: nonce already used')
(False, 'Replay detected: request expired')
```

Experiment 5: Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (RSA and ECDSA) in Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

Python Code

```
import time
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding
from cryptography.hazmat.primitives import hashes

message = b"Digital signature performance test message"

def benchmark_rsa():
```

```

    start = time.time()
    private_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
    keygen_time = time.time() - start
    public_key = private_key.public_key()
    start = time.time()
    signature = private_key.sign(message, padding.PKCS1v15(),
hashes.SHA256())
    sign_time = time.time() - start
    start = time.time()
    public_key.verify(signature, message, padding.PKCS1v15(),
hashes.SHA256())
    verify_time = time.time() - start
    return keygen_time, sign_time, verify_time

def benchmark_ecdsa():
    start = time.time()
    private_key = ec.generate_private_key(ec.SECP256R1())
    keygen_time = time.time() - start
    public_key = private_key.public_key()
    start = time.time()
    signature = private_key.sign(message, ec.ECDSA(hashes.SHA256()))
    sign_time = time.time() - start
    start = time.time()
    public_key.verify(signature, message, ec.ECDSA(hashes.SHA256()))
    verify_time = time.time() - start
    return keygen_time, sign_time, verify_time

rsa_times = benchmark_rsa()
ecdsa_times = benchmark_ecdsa()

print("Algorithm    KeyGen(s)    Sign(s)    Verify(s)")
print(f"RSA          {rsa_times[0]:.5f}    {rsa_times[1]:.5f}
{rsa_times[2]:.5f}")
print(f"ECDSA        {ecdsa_times[0]:.5f}    {ecdsa_times[1]:.5f}
{ecdsa_times[2]:.5f}")

```

Step-by-Step Explanation

Step 1: Import the cryptography library's `rsa`, `ec`, and `padding` modules for key generation and signing, and `hashes` for the SHA-256 digest used inside both signature schemes.

Step 2: Define a fixed test message that both algorithms will sign so the comparison is fair.

Step 3: Write `benchmark_rsa()` which times key generation for a 2048-bit RSA key, then times signing the message with PKCS1v15 padding and SHA-256, then times verifying that signature.

Step 4: Write `benchmark_ecdsa()` which times key generation for an ECDSA key on the SECP256R1 curve, then times signing and verifying the same message using ECDSA with SHA-256.

Step 5: Call both benchmark functions and store their (keygen, sign, verify) timing tuples.

Step 6: Print a formatted table comparing RSA and ECDSA across the three timing metrics.

Step 7: Analyze results: RSA key generation is typically much slower than ECDSA because RSA needs large prime search, while ECDSA generally signs faster and produces much smaller keys and signatures for equivalent security; this makes ECDSA better suited for constrained environments like mobile and IoT, whereas RSA remains attractive for legacy compatibility and systems where verification speed matters more than signing speed.

OUTPUT:

```
PS C:\Users\CHANDU\OneDrive\Documents\crypto codes\ass codes> python exp5.py
Algorithm  KeyGen(s)  Sign(s)  Verify(s)
RSA        0.15579    0.00560  0.00030
ECDSA      0.00070    0.03539  0.00023
```